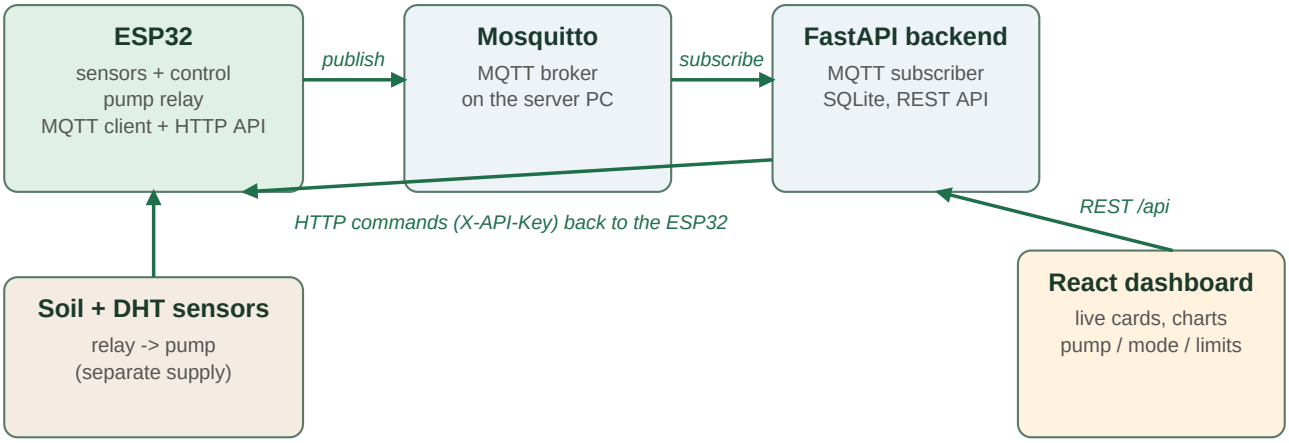
IoT-Based Smart Agriculture and Automated Irrigation System

Viva guide: summary, how it works, what to demo, and likely questions

Say this first (30 seconds)

An ESP32 reads soil moisture, temperature and humidity, and switches a water pump through a relay. The decision to water is made **on the ESP32 itself**, so the field keeps working if Wi-Fi, the broker or the laptop is down. The ESP32 publishes its readings over **MQTT** to a broker; a **FastAPI** backend stores them in **SQLite**; a **React** dashboard shows live values and history and lets the user switch auto/manual mode, run the pump, and change the thresholds through **HTTP/REST** commands that the backend forwards to the ESP32.

Architecture



Telemetry (many small, regular messages) goes over MQTT. Commands go over HTTP because they need an immediate answer: success, or a refusal such as 'pump locked out'.

What is built

- **Firmware** (Arduino C++, ESP32 DevKit V1): sensor reading with calibration, hysteresis pump control, safety lockouts, Wi-Fi + MQTT with automatic reconnect, NTP time, HTTP API protected by an API key.
- **Backend** (FastAPI, paho-mqtt, SQLite): stores readings and events, derives alerts, validates commands and forwards them to the ESP32. Has a demo mode with a simulated device.
- **Dashboard** (React + Vite + TypeScript + Recharts): live cards, soil / temperature / humidity charts with threshold lines and pump-ON shading, controls, alerts, event log, works on a phone.
- **Tests and docs**: 158 C++ checks for the firmware logic (run on a PC), 19 backend tests, integration checklist, API docs, Postman collection, setup guide.

1. Hardware and wiring

Part	Connection	Why / note
ESP32 DevKit V1	USB 5 V power, 3.3 V logic	GPIO is 3.3 V only; never feed 5 V into a GPIO.
Soil moisture sensor (analog)	AOUT -> GPIO34, VCC 3V3, GND	GPIO34 is on ADC1. ADC2 stops working while Wi-Fi is on.
DHT11 (or DHT22)	DATA -> GPIO4, VCC 3V3, GND	Selected by one flag, USE_DHT22. DHT11 +-2 C, DHT22 more accurate.
Relay module (active-low)	IN -> GPIO26, VCC, GND	Switches the pump supply. ON = pin pulled LOW.
Pump	Through the relay, own supply	Never drive a pump from a GPIO. Flyback diode across a DC pump.
Status LED	GPIO2 (on-board)	Blinks with each sensor cycle.
Float switch (optional)	GPIO27 to GND	Not fitted. Flag ENABLE_FLOAT_SWITCH = 0, tank assumed to have water.

Safety points to mention

- Relay is switched **OFF first thing at boot**, before anything else runs.
- Pump has its own supply, the relay only switches it; flyback diode absorbs the voltage spike when the motor switches off.
- Water kept away from the electronics. Mains-powered pumps only under qualified supervision (we used a low-voltage setup).
- Without a float switch, **max run time + cooldown** are the protection against running dry.

2. How the firmware decides

Soil moisture to percent

The soil sensor gives a raw ADC value (0-4095). It is averaged over 10 samples, then mapped linearly between two calibration points (**dry** value in air, **wet** value in water) to 0-100 %. Calibration is done with serial commands (`cal dry`, `cal wet`) and stored in flash (NVS) so it survives reboots. A raw value outside 50-4090 is treated as a **failed sensor** (wire off or shorted), not as 'very dry'.

Automatic control: hysteresis

Pump starts when soil is **below 30 %** and stops when it is **above 60 %** (both adjustable). The gap between the two thresholds prevents the relay from rapidly switching on and off around one value.

Safety lockouts (cannot be bypassed, not even by a manual command)

Lockout	Trigger	Result
Tank empty	Float switch shows no water (only if fitted)	Pump off, alert once, reason LOCKOUT_TANK_EMPTY
Sensor failure	Soil reading invalid, in auto mode	Pump off, reason LOCKOUT_SENSOR_FAIL
Max run time	Pump ran 60 s (adjustable) without stopping	Pump off, then 300 s cooldown blocks all restarts

Other design choices

- **Control logic is separate from hardware** (controller.h): time and sensor values are passed in, so it is unit-tested on a PC with no board attached.
- Timing uses millis() with unsigned subtraction, so it stays correct when the 49-day counter rolls over (there is a test for it).

- Manual mode: the user can switch the pump on or off, but tank, sensor and max-run safeties still apply. Switching mode always turns the pump off first.
- Every pump change carries a **reason code** (AUTO_LOW_MOISTURE, MANUAL_ON, LOCKOUT_MAX_RUNTIME, ...) so the dashboard can explain what happened.
- Thresholds are validated (0 <= on < off <= 100, run 1-3600 s, cooldown 0-86400 s) and saved in flash.

3. Communication: MQTT and HTTP

MQTT topic (prefix farm/field1)	Direction	Content
/sensors	ESP32 -> broker, every 5 s	JSON: soil, soil_raw, temp, hum, tank_ok, pump, mode, reason, sensor_ok, rssi, ts
/events	ESP32 -> broker, on change	JSON: type (pump or alert), message, reason
/status (retained)	ESP32 -> broker	'online' on connect; broker publishes 'offline' via Last Will if the link drops

ESP32 HTTP route (header X-API-Key)	Body	Answers
GET /status	-	200 with full status and thresholds
POST /pump	{"state":"on"|"off"}	200, or 409 (auto mode / tank empty / cooling down)
POST /mode	{"mode":"auto"|"manual"}	200
POST /thresholds	{"on","off","max_run_s","cooldown_s"}	200, or 400 for invalid values; saved to flash

Why both protocols?

- **MQTT** is lightweight publish/subscribe: ideal for a constant stream of small sensor messages, many possible subscribers, and the **Last Will** feature tells us instantly if the device vanished.
- **HTTP/REST** is request/response: a command needs to know right away whether it worked or was refused, which MQTT alone does not give.
- **Reconnect handling**: Wi-Fi retries 10 s doubling to 60 s; MQTT retries 1 s doubling to 60 s with a short socket timeout, so a missing broker never freezes pump control (exponential backoff).
- **Time**: NTP gives real timestamps; until it syncs, ts is 0 and the backend stamps arrival time.
- **Security (prototype level)**: API key on every HTTP route, compared in constant time; secrets live in a gitignored secrets.h and .env files, never in code.

4. Backend and dashboard

Backend (FastAPI)

- Subscribes to the three MQTT topics and writes readings and events into **SQLite** (tables: readings, events, settings).
- REST endpoints: GET /api/latest, /api/history?hours=, /api/events, /api/alerts, /api/health; POST /api/pump, /api/mode, /api/thresholds.
- **Validates commands** (pydantic) before they reach the device, forwards them with the API key, passes a device 409 through, and returns 504 if the ESP32 cannot be reached.
- **Alerts are derived**: DEVICE_OFFLINE (no data for 30 s or Last Will 'offline'), TANK_EMPTY, SENSOR_FAILURE, MAX_RUNTIME.
- History is **down-sampled** into time buckets so a 7-day chart stays fast.
- After a successful command the device's reply is stored immediately, so the dashboard shows the new state at once instead of waiting for the next MQTT message.

- **Demo mode** (DEMO=1) runs a simulated device so the whole dashboard can be shown without hardware.

Dashboard (React)

- Six live cards (soil, temperature, humidity, tank, pump, mode) and a device online/offline badge. If the device or backend is down, values are greyed and labelled 'Last known'.
- Charts for 1 h / 6 h / 24 h / 7 d: soil moisture with dashed ON/OFF threshold lines and shaded pump-ON periods; temperature; humidity; plus a table view.
- Controls: Auto/Manual toggle, pump button (disabled with a visible reason when not allowed), threshold form with validation. Alerts panel and event log with readable reason texts.
- Polls every 4 s (live) and 15 s (history). Status is shown with an icon and text, not colour alone. Works on a phone-width screen.

5. Testing and evidence

What	How	Result
Control logic (hysteresis, lockouts, cooldown, rollover, manual rules)	C++ unit tests on the PC	99 checks pass
MQTT payloads and reconnect backoff	C++ unit tests	20 checks pass
HTTP helpers (JSON, API key, thresholds)	C++ unit tests	39 checks pass
Backend (parsing, storage, alerts, validation, forwarding, 409 / 504)	pytest	19 tests pass
Dashboard	TypeScript build + manual run in demo mode and on a phone	Builds; works
Real hardware	Bench run: DHT, soil sensor, Wi-Fi, MQTT into backend, live dashboard, relay switching	Worked on the bench; fill in your own numbers (see below)

Before the viva: record your own bench numbers (dry value, wet value, relay polarity, a real auto on/off cycle with water, the max-run lockout, a broker restart). A results table is in *docs/report_notes.md*. Examiners trust measured values more than claims.

6. Problems we hit and how they were solved

Good to tell as a story. It shows real debugging, not a copied project.

Problem	Cause	Fix
Soil reading jumped between 0 and 4095	Floating ADC pin / loose jumper wires	Reseated wires; firmware now treats out-of-range readings as a sensor failure and keeps the pump off
DHT showed nan / TIMEOUT	Wrong sensor type in code for the sensor fitted, or poor power	One flag (USE_DHT22) selects the type; last good value is kept for 30 s
False 'tank empty' alert at boot	Control ran before the first sensor reading	Control waits for the first real reading
Dashboard offline though ESP32 was connected	Two Mosquitto brokers: Windows service and a console one; backend and ESP32 used different ones	Stopped and disabled the service; one broker only
MQTT failed after changing Wi-Fi	Laptop got a new IP; Windows treats the network as Public so the firewall rule did not apply	Updated broker IP in secrets.h, firewall rule for all profiles
Relay stayed ON when ESP32 said OFF	Active-low module on 5 V cannot be switched off by a 3.3 V HIGH	OFF now releases the pin (high impedance) so the module's pull-up turns it off

Problem	Cause	Fix
Buttons seemed not to react	Dashboard read the last MQTT message, up to 5 s old; slow polls could overwrite newer state	Backend stores the device's reply to each command immediately; dashboard ignores stale polls
Wi-Fi password pushed to GitHub	Real values were in secrets.example.h	Replaced with placeholders; lesson: keep real secrets only in gitignored files, rotate any leaked password

7. Likely viva questions and short answers

Q. Why does the ESP32 decide, and not the server?

So watering and safety keep working when Wi-Fi, the broker or the laptop fail. The server is only for monitoring and remote control.

Q. Why MQTT for data and HTTP for commands?

MQTT suits a steady stream of small messages and gives Last Will for offline detection. HTTP gives an immediate success or refusal for a command.

Q. What is hysteresis and why use it?

Two thresholds (start below 30 %, stop above 60 %) instead of one. It stops the relay chattering on and off around a single value and protects the pump and relay.

Q. What happens if the soil sensor breaks or is unplugged?

The reading falls outside the valid range (50-4090). The firmware flags a sensor failure, stops the pump and will not start it in auto mode until readings are valid again.

Q. What if the tank is empty?

With a float switch fitted the pump is locked out. We did not fit one, so max run time (60 s) and cooldown (300 s) are the protection. Adding the float switch is one config flag.

Q. What is the MQTT Last Will?

A message the broker publishes for the device if it disconnects without saying goodbye. We set it to 'offline' on the status topic, so the dashboard knows within about 15-30 s.

Q. Why GPIO34 for the soil sensor?

It is on ADC1. ADC2 pins cannot be used for analog reads while Wi-Fi is active. GPIO34 is also input-only, which suits a sensor.

Q. Why is a relay needed? Why a flyback diode?

A GPIO gives about 12 mA at 3.3 V, far too little for a pump. The relay isolates and switches the pump supply. A DC motor is an inductor; when switched off it produces a voltage spike, and the diode across it safely absorbs that.

Q. How is the sensor calibrated?

Hold it in air and type 'cal dry', put it in water to the line and type 'cal wet'. Soil % is a linear map between those two values, saved in flash.

Q. How do you make sure the relay is off at power-up?

The first line in setup() sets the relay off, and the relay pin is released/driven to the off level before sensors or Wi-Fi start.

Q. How did you test it without hardware?

The control logic is pure C++ with time and readings passed in, so 158 unit-test checks run on a PC. The backend has a simulated device (demo mode) and 19 pytest tests.

Q. How is the system secured?

API key on every ESP32 route (constant-time compare), secrets in gitignored files, backend validates every command. It is a LAN prototype: plain HTTP and anonymous MQTT. Next step would be TLS, per-device credentials and a login on the dashboard.

Q. What if two commands conflict (dashboard says pump on, but auto logic says off)?

Pump commands are accepted only in manual mode. In auto mode the controller owns the pump, and the API returns 409 with an explanation.

Q. What are the limits of the DHT11?

+-2 C and +-5 % RH, whole-number steps, and about a 1-2 s minimum between reads. The DHT22 is better and is one flag away.

Q. What does 'sensor_ok' mean?

The soil reading is within range and the DHT has produced a valid value in the last 30 s. The pump logic itself depends on the soil sensor only.

Q. How would you scale this to several zones or fields?

One MQTT topic prefix and one ESP32 per zone (farm/field1, field2, ...); the backend and dashboard filter by zone. Add a zone id to the database.

Q. What would you improve with more time?

TLS and login, a flow or current sensor to detect a dry-running pump, queueing events while offline, OTA firmware updates, a hardware timer on the relay, weather-based watering.

Q. What is the biggest weakness?

Without a float switch there is no real dry-run detection, and a firmware hang with the pump on would leave the relay on. Mitigation: short max run time and, ideally, a hardware relay timer.

8. Demo script (about 5 minutes)

- **Start:** double-click start_all.bat (broker, backend, dashboard). Open the dashboard; badge shows 'Device online'. Open the serial monitor to show the raw output.
- **Live data:** hold the soil sensor in air, then in water; show soil % moving on the card and the chart. Breathe on the DHT to move temperature/humidity.
- **Auto mode:** with the sensor dry (below the ON threshold) the pump turns on, the relay clicks, an event appears; put it in water and it turns off above the OFF threshold. Point at the shaded 'Pump ON' band and threshold lines on the chart.
- **Manual mode:** switch to Manual, press Turn pump ON / OFF. Show that the pump button is disabled in auto mode with the reason displayed.
- **Thresholds:** change ON/OFF in the form, save, show it applied (and survives a reset). Try an invalid one (ON above OFF) to show validation.
- **Safety:** show the max-run lockout (temporarily set max run to 10 s), or unplug the soil sensor signal wire to show SENSOR_FAIL and the pump staying off.
- **Resilience:** stop the broker or backend window; the relay logic keeps working and the dashboard shows 'Last known' values and an offline badge. Restart it and it recovers by itself.

Keep a backup: if hardware misbehaves on the day, run the backend with DEMO=1. The dashboard then shows the same behaviour from the simulated device. Also keep the pump disconnected (relay click only) unless you have tested it with water beforehand.

9. Cheat sheet

Item	Value
Defaults	ON below 30 %, OFF above 60 %, max run 60 s, cooldown 300 s
Valid soil raw range	50 to 4090 (outside = sensor failure)
Publish period	Sensors every 5 s, serial print every 2 s
Offline rule (backend)	No data for more than 30 s, or Last Will 'offline'
Ports	MQTT 1883, ESP32 HTTP 80, backend 8000, dashboard 5173
Topics	farm/field1/sensors, /events, /status

Item	Value
Serial commands	cal dry | cal wet | cal show | cal reset | mode auto|manual | pump on|off | th | th show | help
Tech stack	ESP32 Arduino C++, PubSubClient, MQTT (Mosquitto), FastAPI, SQLite, React + Vite + TypeScript + Recharts
Repository	github.com/Vedant28082005/het_smart_agriculture
Docs in the repo	README, docs/wiring.md, mqtt.md, api.md, integration.md, report_notes.md, setup_server.md

Tip: if you do not know an answer, say what you would check or how you would test it. The troubleshooting stories in section 6 show you can debug.